1. State Definition· We use TypedDict to define the structure of the state.

· In this case, the state has one field: study_plan which is a string.

· This ensures every node in the graph knows what data it will receive and return.

2. Nodes· Each node is a function that takes the current state and returns a new state.

· start_study: appends "I am starting my study session."

· math: appends "Focus on Math today."

· science: appends "Focus on Science today."

· Nodes represent actions or steps in the workflow.

3. Conditional Logic· The function random_subject decides whether to go to the math node or the science node.

· This introduces branching in the workflow, showing how decisions can be modeled.

4. Graph Construction· graph.add_node registers each node function with a name.

· graph.add_edge connects nodes in sequence.

· graph.add_conditional_edges connects a node to multiple possible next nodes based on logic.

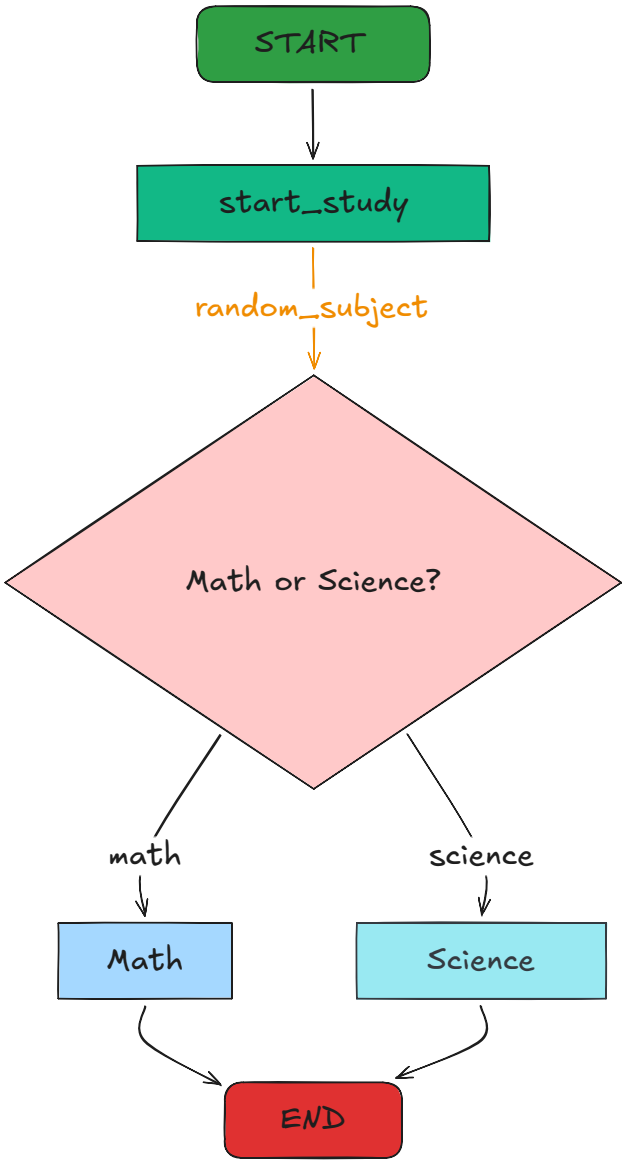
· START and END are special markers for the beginning and end of the workflow.

5. Graph Compilation· graph.compile() turns the abstract design into a runnable agent.

· This agent can be invoked with an initial state to execute the workflow.

6. Visualization· The graph can be visualized as a flowchart.

· This helps in understanding the structure and debugging the workflow.



7. Execution· Running the graph with an initial state such as {"study_plan": "Hello, I am a student."} starts the workflow.

· The state evolves step by step, appending new information at each node.

· The final output shows the complete study plan after passing through the graph.

Main Motive of Writing This Agent

· To learn how to represent processes as graphs of nodes and edges rather than linear code.

· To understand state management: how data flows and changes across steps.

· To practice modular design: each node is independent and reusable.

· To explore conditional branching: workflows can make decisions dynamically.

· To see how visualization makes abstract workflows easier to grasp.

· To connect this idea to real-world applications such as:· Conversational AI agents

- Workflow automation
- Educational apps
- Data pipelines

The main motive is to show how an agent can be built as a graph-based workflow.

Each node represents a step, edges define transitions, and the state carries information forward. This approach is powerful for designing agents that need to handle complex, branching processes in a clear and maintainable way.